

CSCI-4208: Developing Advanced Web Applications

Week 9: Lecture 14

Node Package Manager (npm)

npm - introduction

What is NPM?

NPM is a package manager for Node.js packages, or modules if you like.

www.npmjs.com hosts thousands of free packages to download and use.

The NPM program is installed on your computer when you install Node.js

npm - introduction

What is a Package?

A package in Node.js contains all the files you need for a module.

Modules are JavaScript libraries you can include in your project.

Overview: NPM intro

NPM

npm with its simple structure helped the ecosystem of Node.js proliferate, and now the npm registry hosts over 1,000,000 open source packages you can freely use.

npm - Essential Commands

- npm init
- npm install <module>
- package.json
- npm start

npm: An introduction to the npm package manager

Introduction to npm

npm is the standard package manager for Node.js.

Downloads

npm manages downloads of dependencies of your project.

Installing all dependencies

If a project has a `package.json` file, by running

```
npm install
```

it will install everything the project needs, in the `node_modules` folder, creating it if it's not existing already.

npm: An introduction to the npm package manager

Installing a single package

You can also install a specific package by running

```
npm install <package-name>
```

Updating packages

Updating is also made easy, by running

```
npm update
```

npm will check all packages for a newer version that satisfies your versioning constraints.

You can specify a single package to update as well:

```
npm update <package-name>
```

npm: An introduction to the npm package manager

Versioning

In addition to plain downloads, npm also manages versioning, so you can specify any specific version of a package, or require a version higher or lower than what you need.

Many times you'll find that a library is only compatible with a major release of another library.

Or a bug in the latest release of a lib, still unfixed, is causing an issue.

Specifying an explicit version of a library also helps to keep everyone on the same exact version of a package, so that the whole team runs the same version until the package.json file is updated.

In all those cases, versioning helps a lot, and npm follows the semantic versioning (semver) standard

npm: An introduction to the npm package manager

Running Tasks

The package.json file supports a format for specifying command line tasks that can be run by using

```
npm run <task-name>
```

For example:

```
{  
  "scripts": {  
    "start-dev": "node lib/server-development",  
    "start": "node lib/server-production"  
  }  
}
```

So instead of typing those long commands, which are easy to forget or mistype, you can run

```
npm run start  
npm run start-dev
```

npm - Where does npm install the packages?

When you install a package using npm you can perform 2 types of installation:

- a local install
- a global install

By default, when you type an npm install command, like:

```
npm install lodash
```

the package is installed in the current file tree, under the node_modules subfolder.

As this happens, npm also adds the lodash entry in the dependencies property of the package.json file present in the current folder.

npm - Where does npm install the packages?

A global installation is performed using the `-g` flag:

```
npm install -g lodash
```

When this happens, npm won't install the package under the local folder, but instead, it will use a global location.

Where, exactly?

The `npm root -g` command will tell you where that exact location is on your machine.

npm - How to use a package installed using npm

To use it in your code, you just need to import it into your program using require:

```
const _ = require('lodash')
```

npm - The package.json guide

Introduction

The package.json file is a manifest for your project. It's a central repository of configuration for tools, for example. It's also where npm store the names and versions for all the installed packages.

npm - The package.json guide

The file structure

Here's an example package.json file:

```
{}
```

It's empty! There are no fixed requirements of what should be in a package.json file, for an application. The only requirement is that it respects the JSON format, otherwise it cannot be read by programs that try to access its properties programmatically.

npm - The package.json guide

Properties:

- `version` indicates the current version
- `name` sets the application/package name
- `description` is a brief description of the app/package
- `main` set the entry point for the application
- `private` if set to true prevents the app/package to be accidentally published on npm
- `scripts` defines a set of node scripts you can run
- `dependencies` sets a list of npm packages installed as dependencies
- `devDependencies` sets a list of npm packages installed as development dependencies
- `engines` sets which versions of Node.js this package/app works on
- `browserslist` is used to tell which browsers (and their versions) you want to support

All those properties are used by either npm or other tools that we can use.

npm - uninstalling npm packages

To uninstall a package you have previously installed locally (using `npm install <package-name>` in the `node_modules` folder, run

```
npm uninstall <package-name>
```

from the project root folder (the folder that contains the `node_modules` folder).

Using the `-S` flag, or `--save`, this operation will also remove the reference in the `package.json` file.

If the package is installed globally, you need to add the `-g` / `--global` flag:

```
npm uninstall -g <package-name>
```

npm - global or local packages

The main difference between local and global packages is this:

- local packages are installed in the directory where you run `npm install <package-name>`, and they are put in the `node_modules` folder under this directory
- global packages are all put in a single place in your system (exactly where depends on your setup), regardless of where you run `npm install -g <package-name>`

In your code you can only require local packages:

```
require('package-name')
```

npm - global or local packages

So when should you install in one way or another?

In general, all packages should be installed locally. This makes sure you can have dozens of applications in your computer, all running a different version of each package if needed. All projects have their own local version of a package, even if this might appear like a waste of resources, it's minimal compared to the possible negative consequences.

A package should be installed globally when it provides an executable command that you run from the shell (CLI), and it's reused across projects.

npm - npm dependencies

When you install an npm package using `npm install <package-name>`, you are installing it as a dependency.

The package is automatically listed in the `package.json` file, under the dependencies list.

if you type `npm install` and the folder contains a `package.json` file, they are installed

npm - The npx Node.js Package Runner

npx lets you run code built with Node.js and published through the npm registry.

Easily run local commands

Running `npx commandname` automatically finds the correct reference of the command inside the `node_modules` folder of a project, without needing to know the exact path, and without requiring the package to be installed globally and in the user's path.

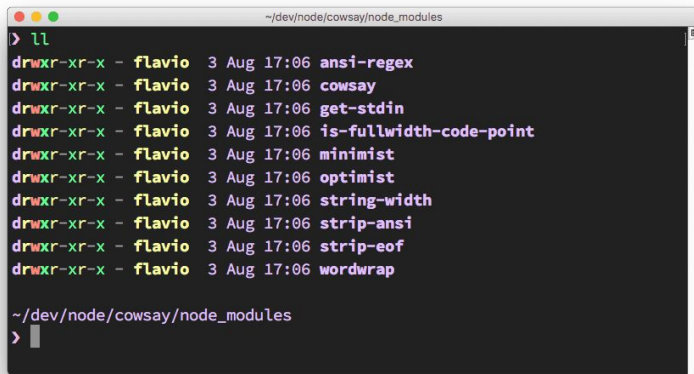
npm - npx

What if your package is an executable?

In this case, it will put the executable file under the `node_modules/.bin/` folder.

One easy way to demonstrate this is `cowsay`. The `cowsay` package provides a command line program that can be executed to make a cow say something (and other animals as well 🦊).

When you install the package using `npm install cowsay`, it will install itself and a few dependencies in the `node_modules` folder:



```
~/dev/node/cowsay/node_modules
> ll
drwxr-xr-x - flavio  3 Aug 17:06 ansi-regex
drwxr-xr-x - flavio  3 Aug 17:06 cowsay
drwxr-xr-x - flavio  3 Aug 17:06 get-stdin
drwxr-xr-x - flavio  3 Aug 17:06 is-fullwidth-code-point
drwxr-xr-x - flavio  3 Aug 17:06 minimist
drwxr-xr-x - flavio  3 Aug 17:06 optimist
drwxr-xr-x - flavio  3 Aug 17:06 string-width
drwxr-xr-x - flavio  3 Aug 17:06 strip-ansi
drwxr-xr-x - flavio  3 Aug 17:06 strip-eof
drwxr-xr-x - flavio  3 Aug 17:06 wordwrap

~/dev/node/cowsay/node_modules
> 
```

npm - npx

There is a hidden `.bin` folder, which contains symbolic links to the cowsay binaries:

```
~/dev/node/cowsay/node_modules/.bin
> ll
lrwxr-xr-x 16 flavio  3 Aug 17:06 cowsay -> ../cowsay/cli.js
lrwxr-xr-x 16 flavio  3 Aug 17:06 cowthink -> ../cowsay/cli.js

~/dev/node/cowsay/node_modules/.bin
>
```

How do you execute those? You can of course type `./node_modules/.bin/cowsay` to run it.

You just run:

```
npx cowsay
```

```
~/dev/node/cowsay/node_modules/.bin
> npx cowsay take me out of here

  < take me out of here >
  -----
      \   ^__^
       \  (oo)\_______
          (__)\       )\/\
             ||----w |
             ||     ||

~/dev/node/cowsay/node_modules/.bin
>
```

The End.

COMING SOON... TO A MOODLE NEAR YOU!

Labs

Web Server in Node & npm

Homework

TBD